OOP Concepts

7.3 Abstraction

Intended Learning Outcomes

After completing the lesson, students should be able to:

- Describe the concepts of abstraction
- Create `abstract` class
- Create class that implements multiple `interface`s in Java

Abstraction Concept

- focuses on
  - hiding the internal implementation details of a class
  - only exposing the necessary functionalities to the users
- Achieved by
  - Abstract class
  - Interface

Benefits of Abstraction

1. Reduces complexity
2. Allows focusing on interactions at a higher level
3. Facilitates code reusability and scalability
4. Isolates the impact of changes made to the code,
   - as they do not affect the interface
5. Promotes the separation of concerns,
   - separating a program into distinct sections

Abstract Class

- cannot be instantiated, but other classes are derived from it.
- serves as a base class (superclass) for other classes.
- represents the generic or abstract form
  - of all the classes that are derived from it.
- Syntax:

```
public abstract class AbstractClassName
```

Student.java > Student > Student(String, String, int)

```java
/**
 *  The Student class is an abstract class that holds
 *  general data about a student. Classes representing
 *  specific types of students should inherit from
 *  this class.
 */

public abstract class Student
{
    private String name;       // Student name
    private String idNumber;   // Student ID
    private int yearAdmitted; // Year student was admitted

```

Abstract Methods

- must be overridden in a subclass.
- a method that appears in a superclass,
  - but expects to be overridden in a subclass.
- has only a header and no body.
- Syntax:

```
AccessSpecifier abstract ReturnType MethodName(ParameterList);
```

- Example:

```
public abstract int getRemainingHours();
```

  - Student.java, CompSciStudent.java, CompSciStudentDemo.java

Student.java > ...

```java
8   public abstract class Student
26
27      /**
28       *  toString method
29       */
30
31      public String toString()
32      {
33          String str;
34
35          str = "Name: " + name +
36                "\nID Number: " + idNumber +
37                "\nYear Admitted: " + yearAdmitted;
38
39          return str;
40      }
41
42      /**
43       *  The getRemainingHours method is abstract.
44       *  It must be overridden in a subclass.
45       */
46
47      public abstract int getRemainingHours();
48  }
```

Abstract Methods

- used to ensure that a subclass implements the method.
- Any class that contains an abstract method
  - is automatically an abstract class.
- If a subclass fails to override an abstract method,
  - a compiler error will result.

Abstract Class Exercise

- Define an abstract class called `Shape`.
- `Shape` should have at least two abstract methods:
  - `area()` and `perimeter()` returning double values.
- Create two concrete classes `Circle` and `Rectangle` that extend `Shape`.
- Implement the abstract methods in the concrete classes.
- Each shape should have properties relevant to calculating area and perimeter
  - radius for the circle,
  - length, and width for the rectangle).
- Include a constructor in each concrete class to set the properties.
- Write a `main` method to instantiate each shape, calculate, and print out the area and perimeter.

Multiple inheritance vs Interface

- Java does not support multiple inheritance of classes.
  - avoid the complexity and ambiguity that multiple inheritance can cause,
- Interface in Java
  - workaround to achieve multiple inheritance-like behavior

Interfaces

- similar to an abstract class that has all abstract methods.
  - cannot be instantiated, and
  - **ALL** of the methods listed in an interface must be written elsewhere.
- Purpose: specify behavior for other classes.
- looks like a class except
  - keyword `interface` is used (not `class`)
  - methods has only headers end with semicolons
    - have no bodies,

Interfaces

- Syntax:

```
public interface InterfaceName{
    ReturnType methodName(ParameterType para);
}
```

- All methods specified by an interface are public by default.
- A class can implement one or **more** interfaces.
- Method bodies exist only for `default` methods and `static` methods.

`default` methods in interface (Java 8+)

- allows a class to inherit certain behaviors from multiple interfaces.
- interfaces can evolve by adding new methods
  - without breaking existing implementations.
  - useful for libraries or APIs that require extending functionality
  - without impacting existing clients.

Example ComparableShape

```java
public interface ComparableShape extends Comparable<ComparableShape> {

    // Abstract method to calculate the area of the shape
    double calculateArea();

    // Default compareTo method comparing shape areas
    @Override
    default int compareTo(ComparableShape otherShape) {
        double myArea = this.calculateArea();
        double otherArea = otherShape.calculateArea();

        return Double.compare(myArea, otherArea);
    }
}
```

`static` methods in interface

- can be used to provide utility functions related to the interface
- useful for methods that don't require an object state
  - but are closely related to the interface's domain.

Interfaces

- If a class implements an interface, it uses the `implements` keyword in the class header.
- Example:

```
public class FinalExam3 extends GradedActivity implements Relatable
```

- Example:
  - GradedActivity.java
  - Relatable.java
  - FinalExam3.java
  - InterfaceDemo.java

Fields in Interfaces

- all fields in an interface are treated as `final` and `static`.
- must provide an initialization value (because of final)
- Any class that implements the interface has access to those variables.

Example of fields

```java
public interface Day {
    int MONDAY = 1;
    int TUESDAY = 2;
    int WEDNESDAY = 3;
    int THURSDAY = 4;
    int FRIDAY = 5;
    int SATURDAY = 6;
    int SUNDAY = 7;
    // ...
}
```

```java
public static String getDayName(int dayNumber) {
    switch (dayNumber) {
        case Day.MONDAY: return "Monday";
        case Day.TUESDAY: return "Tuesday";
        case Day.WEDNESDAY: return "Wednesday";
        case Day.THURSDAY: return "Thursday";
        case Day.FRIDAY: return "Friday";
        case Day.SATURDAY: return "Saturday";
        case Day.SUNDAY: return "Sunday";
        default: return "Invalid day number";
    }
}
```

Implementing Multiple Interfaces

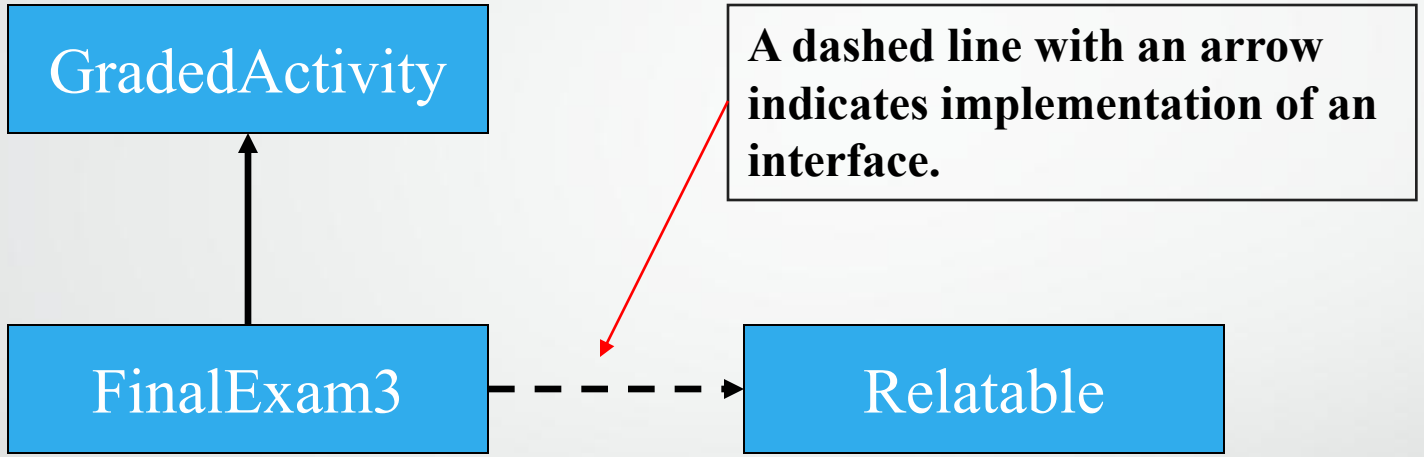
- Java allows a class to implement multiple interfaces.
  - A class can be extended from only one superclass.
- When implementing multiple interfaces,
  - it must provide the methods specified by **ALL** of them.

Implements multiple interfaces

- To specify multiple interfaces,
- simply list the names of the interfaces,
- separated by commas,
- after the implements key word.

```
public class MultiInterfaceClass implements Interface1,
                                            Interface2,
                                            Interface3
```

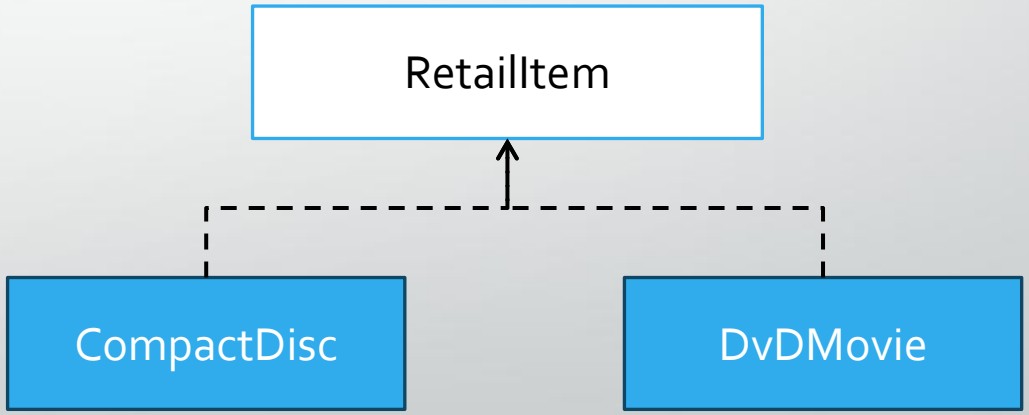
Interfaces in UML




```
public class FinalExam3 extends GradedActivity implements Relatable
```

Polymorphism with Interfaces

- Java allows you to create reference variables of an interface type.
- An interface reference variable can reference any object that implements that interface, regardless of its class type.
- another example of polymorphism.
- Example:
  - RetailItem.java
  - CompactDisc.java
  - DvdMovie.java
  - PolymorphicInterfaceDemo.java



Example Reference Variable of Interface

```
/**
 *  The showPrice method displays the price
 *  of the RetailItem object that is passed
 *  as an argument.
 */

private static void showPrice(RetailItem item)
{
   System.out.printf("Price: $%,.2f\n", item.getRetailPrice());
}
```

See: PolymorphicInterfaceDemo.java

Interface inheritance

- When a class implements an interface,
  - an inheritance relationship is established.
- Example
  - a `CompactDisc` object is a `RetailItem`
  - a `DvdMovie` object is a `RetailItem`.

Polymorphism with Interfaces

- A reference to an interface can point to any class that implements that interface.
- CANNOT create an instance of an interface.

```
RetailItem item = new RetailItem(); // ERROR!
```

Type Casting for interface reference

- When an interface variable references an object:
  - only the methods declared in the interface are available,
- explicit type casting is required
  - to access the other methods of an object
  - referenced by an interface reference.

Example of Interface Type Casting

```
RetailItem item1 = new CompactDisc("Greatest Hits",
                                   "Joe Looney Band",
                                   18.95);
CompactDisc cd = (CompactDisc) item1;
// Display the CD's title.
System.out.println("Item #1: " +
                   cd.getTitle());
```

Type compatibility checking (`instanceof`)

- We can check Type compatibility before casting
- Syntax: `object instanceof TargetType`
- Example:

```
if (item2 instanceof DvdMovie)
        movie = (DvdMovie) item2;
```

Multiple Inheritance exercise

- Write a Java class (Device) that represents an electronic device that implement the Switchable and Identifiable interfaces.
- Requirements:
  - Have a private attribute id of type String.
  - Implement the Switchable interface with methods turnOn and turnOff.
  - Implement the Identifiable interface with the method getId.
  - Provide a constructor to initialize the id.
  - Print appropriate messages when the turnOn and turnOff methods are called.

Sample Output

```
Device 12345 is now ON.
Device 12345 is now OFF.
Device ID: 12345
```